Extractor de facturas FEL con LLM

Angel Esteban Esquit Hernández Javier Eduardo España Pacheco
CC3092 Deep Learning y Sistemas Inteligentes

Proyecto 3 – Entrega: 6 de noviembre de 2026

Índice

2.1. Problema y usuario 2
2.2. Producto y arquitectura 2
2.3. Datos 2
2.3.1. FEL sintético 2
2.3.2. Dataset público adicional 3
2.4. Modelos y técnica 3
2.5. Evaluación 3
2.5.1. Resultados FEL, 45 documentos 3
2.5.2. Resultados públicos, 15 documentos 3
2.6. Experiencia de usuario 4
2.6.1. Prueba de usabilidad 4
2.7. Riesgos, privacidad y limitaciones 5
2.8. Conclusiones y trabajo futuro 5
2.9. Reproducibilidad y repositorio 5

2.1. Problema y usuario

Las pymes y los equipos contables reciben facturas en PDF o imagen con layouts, idiomas, monedas y etiquetas diferentes. La digitación manual en una hoja de cálculo es repetitiva y las reglas por proveedor no escalan. El producto recibe una factura y devuelve un JSON/CSV revisable con los campos principales de una factura FEL.

El usuario objetivo es un contador o auxiliar administrativo que necesita registrar facturas con rapidez, pero debe conservar revisión humana antes de usar los datos en contabilidad oficial. El sistema no sustituye una validación tributaria ni una aprobación contable.

2.2. Producto y arquitectura

El flujo del producto es:

1. El usuario sube un PDF, PNG, JPG o WEBP desde Streamlit.
2. El extractor lo convierte en una entrada multimodal y solicita una salida estructurada `Factura` con Pydantic.
3. Los validadores revisan IVA, sumas, líneas, NIT, fecha y UUID.
4. Si hay inconsistencias, se puede hacer un único reintento con los problemas explícitos.
5. La interfaz muestra campos y advertencias, y permite descargar JSON y CSV.

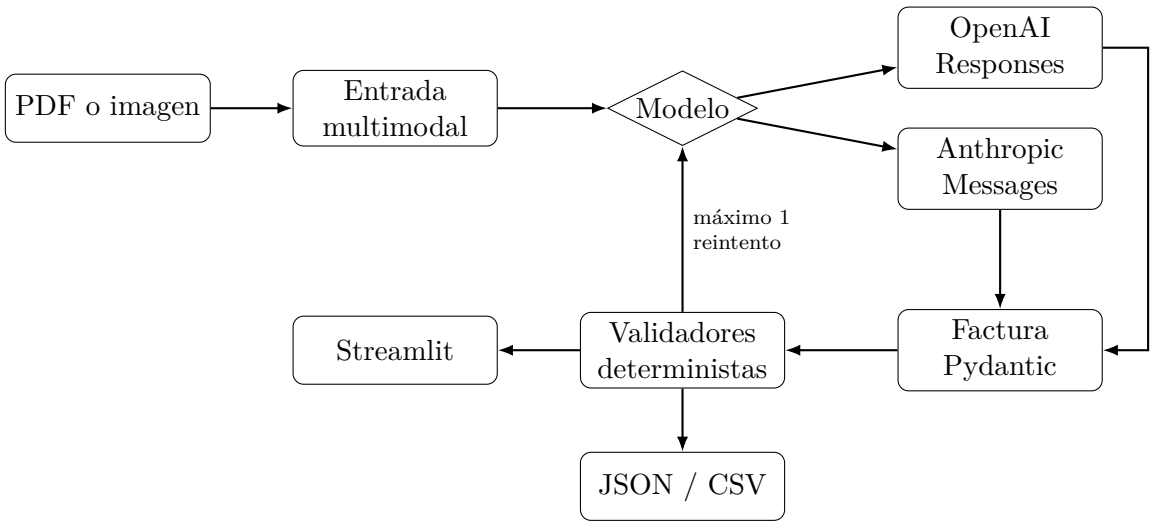


Figura 1: Arquitectura del extractor.

No se usa RAG ni embeddings: la tarea es extracción de un documento individual y no recuperación de conocimiento. La generación se limita a 2.000 tokens; las comparaciones de este informe usan cero reintentos para controlar costo.

2.3. Datos

2.3.1. FEL sintético

El conjunto principal contiene 45 documentos generados con semilla 2026 por `scripts/make_synthetic.py`. Incluye cuatro layouts, PDFs e imágenes, fechas en tres formatos, una a ocho líneas, ruido, rotación, desenfoque y facturas de pequeño contribuyente sin IVA.

Durante la revisión se corrigió un error de etiquetado: seis facturas sin IVA tenían `subtotal` igual al total aunque el documento no lo mostraba. El gold corregido usa `subtotal: null` y se regeneraron los seis JSON afectados. La corrección se documenta como error de etiquetado, no como ajuste de una predicción.

2.3.2. Dataset público adicional

Se integraron 15 fixtures sintéticas del Synthetic Shopify Invoice Test Pack, con licencia CC BY 4.0. Cada caso conserva PDF y PNG y tiene una referencia convertida al esquema local. Incluye multipágina, árabe, bilingüe, descuentos, flete, duty, mala calidad y tablas densas.

Este conjunto es una prueba de robustez y no representa facturas FEL guatemaltecas: sus campos fiscales, monedas y estructura comercial son distintos. No se presentan como facturas reales ni como validación tributaria.

2.4. Modelos y técnica

Se compararon tres modelos multimodales con salida estructurada:

- `gpt-4o-mini`, mediante OpenAI Responses API.
- `claude-haiku-5-5`, mediante Anthropic Messages API.
- `claude-sonnet-5-5`, mediante Anthropic Messages API.

La técnica combina prompting, esquema Pydantic, validación determinista y retroalimentación opcional. No se hizo fine-tuning. El prompt v1 es mínimo; v2 especifica nulos, normalización, fechas, montos y defensa contra instrucciones incrustadas en el documento.

2.5. Evaluación

Se midió exactitud por campo, porcentaje de documentos completamente correctos, salida válida, latencia, tokens y costo estimado. El baseline es regex sobre texto de PDF y no procesa imágenes.

2.5.1. Resultados FEL, 45 documentos

Cuadro 1: Comparación principal sin reintentos.

Sistema	Exactitud	Docs. 100 %	Latencia	Costo/doc.
Regex	23,25 %	0 %	–	USD 0
GPT-4o-mini	90,77 %	15,56 %	4,923 s	USD 0,00147
Claude Haiku 5.5	100 %	100 %	5,305 s	USD 0,00076
Claude Sonnet 5.5	98,63 %	82,22 %	3,958 s	USD 0,0113

El costo total de las tres corridas LLM fue aproximadamente USD 0,6084. Haiku fue la mejor relación precisión/costo en este conjunto. Sonnet no justificó su costo adicional.

2.5.2. Resultados públicos, 15 documentos

Cuadro 2: Comparación sobre el dataset público.

Sistema	Exactitud	Docs. 100 %	Latencia	Costo/doc.
GPT-4o-mini	72,31 %	0 %	3,657 s	USD 0,00155
Claude Haiku 5.5	83,08 %	6,67 %	4,120 s	USD 0,00078
Claude Sonnet 5.5	91,28 %	33,33 %	3,601 s	USD 0,01162

En este conjunto Sonnet obtuvo el mejor resultado, seguido por Haiku y GPT. Las principales diferencias aparecen en nombres, NIT y líneas, porque el dataset usa un esquema comercial internacional distinto al FEL local. Esto confirma que la ventaja de Haiku en el conjunto FEL no es universal: depende del dominio y de la distribución de los documentos.

2.6. Experiencia de usuario

La aplicación Streamlit ofrece:

- Selector de modelo y reintentos de corrección.
- Validación de la clave del proveedor seleccionado.
- Límite de archivo de 10 MB y validación de extensión.
- Tabla de campos generales y líneas.
- Advertencias de validadores para revisión humana.
- Descarga de JSON y CSV.
- Latencia, tokens, costo estimado e intentos visibles.



Figura 2: Pantalla inicial: selector de modelo, aviso de privacidad y carga de la factura (PDF, PNG, JPG o WEBP).

La latencia de la Figura 2 (53,8 s) es una sola medición de la interfaz y es mucho mayor que el promedio de 3,96 s de Sonnet en la evaluación por lotes; puede deberse a la primera llamada o a la red, y no se tomó como representativa.

La interfaz no debe usarse como aprobación automática: el resultado siempre requiere revisión cuando hay advertencias o cuando el documento contiene conceptos que el esquema no modela, como descuentos, flete o duty.

2.6.1. Prueba de usabilidad

Se pidió a personas ajenas al grupo, sin ayuda, que (1) subieran una factura y obtuvieran el JSON o CSV, (2) identificaran qué campo quedó marcado para revisión y por qué, y (3) corrigieran un valor. Se registró si completaron cada tarea, el tiempo, dónde dudaron, una calificación de facilidad de 1 a 5 y un comentario.

Cuadro 3: Resultados de la prueba de usabilidad (completar con datos observados).

Tester	Tarea 1	Tarea 2	Tarea 3	Tiempo	Facilidad (1–5)	Comentario

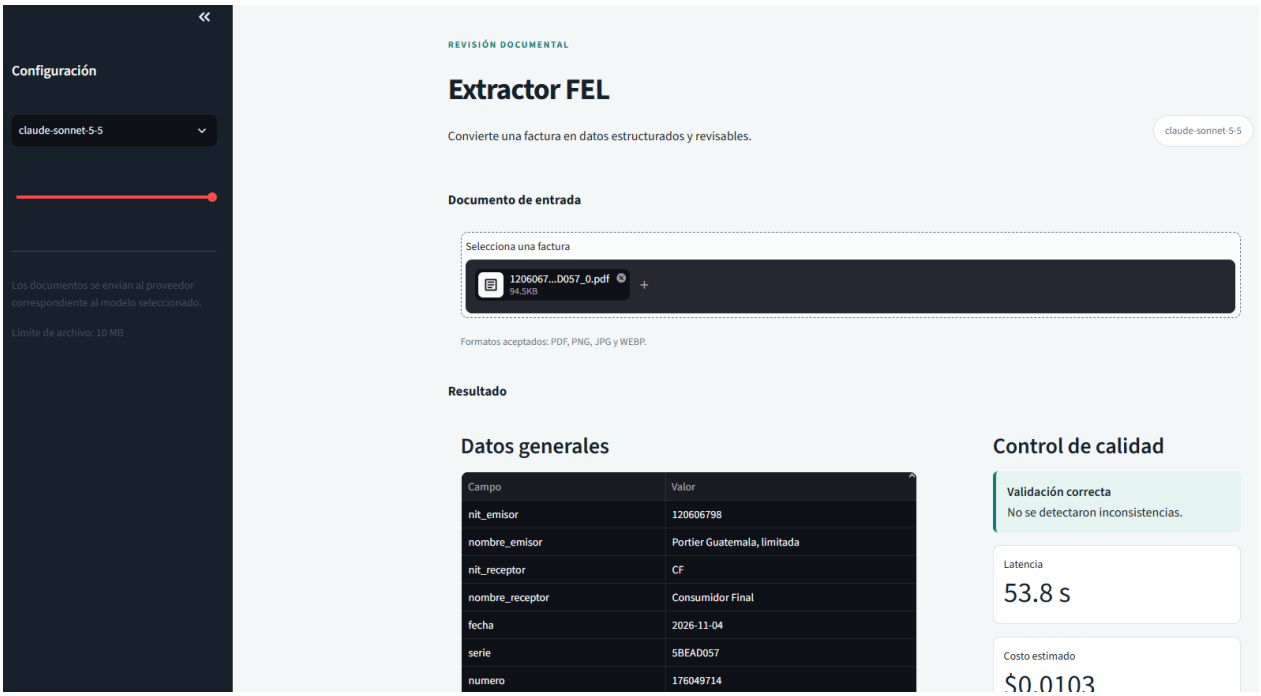


Figura 3: Factura cargada y resultado: datos generales extraídos, control de calidad ("Validación correcta"), latencia y costo estimado.

2.7. Riesgos, privacidad y limitaciones

Los documentos se envían al proveedor correspondiente al modelo seleccionado. No se deben subir facturas que no puedan compartirse con OpenAI o Anthropic. Las claves viven únicamente en `.env`, excluido de Git.

El documento se trata como dato y el prompt ordena ignorar instrucciones contenidas en la factura. Esto reduce prompt injection, pero no elimina errores de interpretación. Los validadores detectan inconsistencias, no garantizan exactitud fiscal.

El conjunto principal es sintético y fue creado por el mismo equipo que diseñó el extractor. El conjunto público también es sintético y no reemplaza datos reales anonimizados del flujo de trabajo. El algoritmo del NIT, los campos FEL y el supuesto de IVA deben contrastarse con documentación oficial antes de producción.

2.8. Conclusiones y trabajo futuro

El producto es funcional y reproducible para demostración y evaluación controlada. Haiku 5.5 es la configuración recomendada por su resultado y costo. GPT-4o-mini es una alternativa de bajo costo, aunque perdió precisión en el conjunto FEL. Sonnet ofrece buen desempeño, pero con un costo aproximadamente quince veces mayor que Haiku en esta corrida.

El siguiente trabajo es añadir facturas reales anonimizadas con permiso, revisión cruzada de sus gold, prueba de usabilidad con usuarios externos, capturas, video de demostración y validación formal de supuestos tributarios. Esos elementos no se deben inventar a partir de datasets sintéticos.

2.9. Reproducibilidad y repositorio

Repositorio de GitHub: `https://github.com/Javier-Espana/PRY3-DL`

```
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt
cp .env.example .env
.venv/bin/python -m pytest -q
.venv/bin/python -m streamlit run src/app.py
```

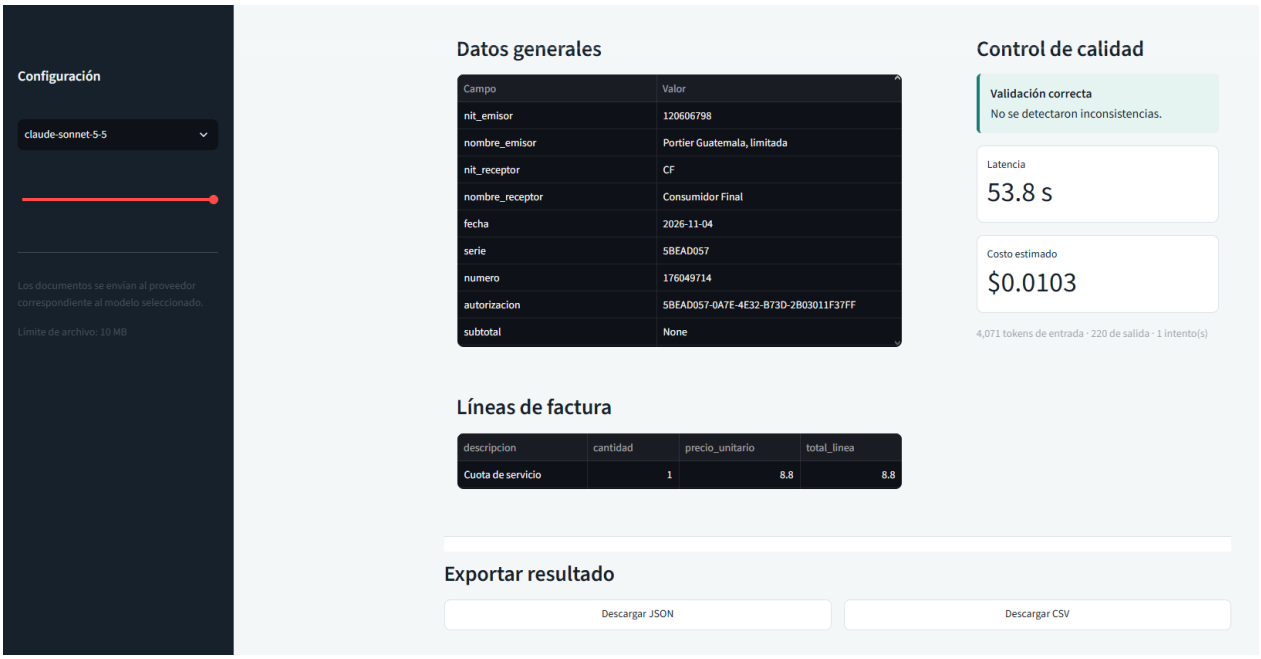


Figura 4: Resto del resultado: líneas de factura, tokens e intentos, y botones para exportar JSON y CSV. El campo `subtotal` aparece como `None` porque el documento no lo muestra.

Para repetir las métricas FEL:

```
.venv/bin/python -m src.evaluate --system regex
.venv/bin/python -m src.evaluate --system llm --model gpt-4o-mini --prompt v2 --retries 0
.venv/bin/python -m src.evaluate --system llm --model claude-haiku-5-5 --prompt v2 --
    retries 0
.venv/bin/python -m src.evaluate --system llm --model claude-sonnet-5-5 --prompt v2 --
    retries 0
```

Para el dataset público:

```
.venv/bin/python -m src.evaluate --system llm --model claude-haiku-5-5 \
  --prompt v2 --retries 0 --docs data/public/shopify/docs \
  --gold data/public/shopify/gold --run-name public_shopify
```

Referencias

[1] SalorWorks, *Synthetic Shopify Invoice Test Pack*, CC BY 4.0, 2026. https://github.com/SalorWorks/shopify-invoice-test-pack

[2] M. Limam, M. Dhiaf y Y. Kessentini, *FATURA Dataset*, Zenodo, 2023. https://zenodo.org/records/8261508